

JavaScript Placement & MERN Master Notebook

Mahil

Contents

1. JAVASCRIPT BASICS & DATA TYPES	8
1. What is it?	8
2. Why does it exist?	8
3. JavaScript vs Java	8
4. JS Engine & Interpreted/JIT	8
5. Variables & Constants	8
6. Primitive Data Types	9
7. Reference Types	9
8. Primitive vs Reference — Comparison Table	9
9. <code>null</code> vs <code>undefined</code>	9
10. <code>NaN</code>	10
11. <code>Symbol</code> & <code>BigInt</code> (brief — rarely core to placement interviews)	10
12. Mutable vs Immutable	10
Interview Questions	10
Interview Trap	10
Quick Revision	10
2. VARIABLES, SCOPE & HOISTING	11
1. What is it?	11
2. Why does it exist?	11
3. Types of Scope	11
4. <code>var</code> vs <code>let</code> vs <code>const</code>	11
5. Hoisting — Output Question	12
6. Temporal Dead Zone (TDZ)	12
Interview Definition	12
Interview Questions	12
Interview Trap	12
Real Project Usage	13
Quick Revision	13
3. OPERATORS & TYPE COERCION	14
1. What is it?	14
2. Why does it exist?	14
3. Implicit vs Explicit Coercion	14
4. <code>==</code> vs <code>===</code>	14
5. Truthy / Falsy Values	14
6. “JavaScript Weirdness” — Interview Favorites	15

Interview Definition	15
Interview Questions	15
Interview Trap	15
Real Project Usage	15
Quick Revision	15
4. FUNCTIONS	16
1. What is it?	16
2. Why does it exist?	16
3. Types of Functions	16
4. Default & Rest Parameters	16
5. Why are functions “first-class citizens”?	17
Interview Definition	17
Interview Questions	17
Interview Trap	17
Real Project Usage	17
Quick Revision	17
5. this KEYWORD	18
1. What is it?	18
2. Why does it exist?	18
3. THIS — Interview Cheat Sheet	18
4. Examples	18
5. call, apply, bind	19
Output Question	19
Interview Definition	19
Interview Questions	19
Interview Trap	19
Real Project Usage	19
Quick Revision	19
6. CLOSURES	21
1. What is it?	21
2. Why does it exist?	21
3. Basic Example — Counter	21
4. Function Factories	21
5. Closures inside Loops — Classic Trap	22
Interview Definition	22
Interview Questions	22
Interview Trap	22
Real Project Usage	22
Quick Revision	22
7. ARRAYS & ARRAY METHODS	24
1. What is it?	24
2. Creating & Basic Operations	24
3. Array Methods — Full Table	24
4. Examples	25
Interview Questions	25

Interview Trap	25
Real Project Usage	25
Quick Revision	25
8. OBJECTS & OBJECT METHODS	26
1. What is it?	26
2. Creating & Accessing	26
3. Nested Objects	26
4. Object Methods	26
5. Object Destructuring (preview — full detail in Ch. 9)	26
Interview Questions	27
Interview Trap	27
Real Project Usage	27
Quick Revision	27
9. DESTRUCTURING, SPREAD & REST	28
1. What is it?	28
2. Spread vs Rest — Key Difference	28
3. Array & Object Destructuring	28
Interview Questions	29
Interview Trap	29
Real Project Usage	29
Quick Revision	29
10. STRINGS & STRING METHODS	30
1. What is it?	30
2. Common Methods	30
Interview Trap	30
Real Project Usage	30
Quick Revision	30
11. DOM & BROWSER JAVASCRIPT	31
1. What is it?	31
2. Common Operations	31
Interview Questions	31
Interview Trap	31
Real Project Usage	31
Quick Revision	32
12. EVENTS & EVENT HANDLING	33
1. What is it?	33
2. Event Bubbling vs Capturing	33
3. Event Delegation	33
4. preventDefault() vs stopPropagation()	33
Interview Questions	33
Interview Trap	34
Real Project Usage	34
Quick Revision	34
13. CALLBACKS	35

1. What is it?	35
2. Why does it exist?	35
3. Callback Hell	35
Interview Questions	35
Interview Trap	36
Real Project Usage	36
Quick Revision	36
14. ASYNCHRONOUS JAVASCRIPT	37
1. What is it?	37
2. Why does it exist?	37
3. The Progression	37
Interview Definition	37
Quick Revision	37
15. PROMISES	38
1. What is it?	38
2. Why does it exist?	38
3. Promise States	38
4. Basic Syntax	38
5. Fixing Callback Hell with Promise Chaining	38
6. Promise Combinators	39
Interview Definition	39
Interview Questions	39
Interview Trap	39
Real Project Usage	39
Quick Revision	39
16. ASYNC/AWAIT	41
1. What is it?	41
2. Basic Syntax	41
3. How it works	41
4. Sequential vs Parallel Await (common interview trap)	41
Interview Definition	41
Interview Questions	42
Interview Trap	42
Real Project Usage	42
Quick Revision	42
17. EVENT LOOP (Most important async topic for interviews)	43
1. What is it?	43
2. The Components	43
3. The Golden Rule	43
4. Classic Output Question — Walkthrough	44
5. More Output Practice	44
Interview Definition	44
Interview Questions	45
Interview Trap	45
Real Project Usage	45

Quick Revision	45
18. MICROTASKS & MACROTASKS	46
1. What is it?	46
2. Table	46
Interview Trap	46
Quick Revision	46
19. ERROR HANDLING	47
1. What is it?	47
2. try/catch/finally	47
3. Handling Async Errors	47
4. Custom Errors	47
Interview Questions	48
Interview Trap	48
Real Project Usage	48
Quick Revision	48
20. ES6+ FEATURES (Quick Reference)	49
Optional Chaining vs Nullish Coalescing — Trap	49
for...of vs for...in	49
Map vs Object	50
Interview Questions	50
Interview Trap	50
Real Project Usage	50
Quick Revision	50
21. MODULES	51
1. What is it?	51
2. ES Modules (used in React/modern JS)	51
3. CommonJS (used traditionally in Node.js)	51
Interview Questions	51
Interview Trap	51
Real Project Usage	52
Quick Revision	52
22. PROTOTYPES & PROTOTYPE CHAIN	53
1. What is it?	53
2. Why does it exist?	53
3. <code>prototype</code> vs <code>__proto__</code>	53
4. Prototype Chain Diagram	54
5. Prototypal Inheritance	54
Interview Definition	54
Interview Questions	54
Interview Trap	54
Real Project Usage	54
Quick Revision	54
23. CLASSES & OOP IN JAVASCRIPT	56
1. What is it?	56

2. Basic Syntax	56
3. Inheritance — extends & super	56
4. OOP Pillars in JS	56
Interview Definition	57
Interview Questions	57
Interview Trap	57
Real Project Usage	57
Quick Revision	57
24. JSON, FETCH API & HTTP BASICS	59
1. What is it?	59
2. JSON Methods	59
3. HTTP Methods	59
4. Common HTTP Status Codes	59
5. Fetch Examples	60
Interview Questions	60
Interview Trap	60
Real Project Usage	60
Quick Revision	60
25. JAVASCRIPT CONCEPTS YOU MUST KNOW BEFORE REACT	61
Example — Rendering a List	61
Example — Immutable State Update	62
Example — Data Fetching	62
Interview Questions	62
Interview Trap	62
Quick Revision	62
26. JAVASCRIPT FOR NODE.JS/EXPRESS	64
Key Concepts in Backend JS	64
Middleware Concept	64
Error-Handling Middleware	65
Interview Questions	65
Interview Trap	65
Real Project Usage	65
Quick Revision	65
27. JAVASCRIPT CONCEPTS USED IN MERN PROJECTS — FULL DATA FLOW	66
JS Concepts Involved at Every Stage	66
Interview Questions (Project-Based)	67
Quick Revision	67
28. JAVASCRIPT + DSA/CODING PATTERNS	68
JS Syntax Needed for DSA	68
1. Frequency Map Pattern	68
2. Two Pointer Pattern	69
3. Sliding Window Pattern	69
4. Hashing Pattern	69
5. Stack Pattern	70
6. Queue Pattern	70

7. Binary Search Pattern	70
8. Prefix Sum Pattern	71
9. Recursion Pattern	71
10. Sorting Pattern	71
Interview Questions	71
Quick Revision	72
29. JAVASCRIPT INTERVIEW QUESTION BANK	73
Beginner (18)	73
Intermediate (22)	73
Advanced (15)	74
30. JAVASCRIPT OUTPUT-BASED QUESTIONS	76
Level 1 — Basic	76
Level 2 — Scope/Hoisting	76
Level 3 — Functions/Closures	77
Level 4 — <code>this</code>	77
Level 5 — Async/Promises	78
Level 6 — Event Loop (multi-step)	79
Bonus — Mixed Traps	80
31. COMMON JAVASCRIPT INTERVIEW TRAPS — “TRAPS INTERVIEWERS LOVE”	81
32. FINAL JAVASCRIPT CHEAT SHEET (20–30 min revision)	83
33. JAVASCRIPT — 1 DAY BEFORE INTERVIEW	86
MUST KNOW	86
SHOULD KNOW	86
IF TIME PERMITS	86
50 QUESTIONS I MUST BE ABLE TO ANSWER	86
20 OUTPUT QUESTIONS I MUST BE ABLE TO SOLVE	86
10 CODING PATTERNS I MUST REMEMBER	87

1. JAVASCRIPT BASICS & DATA TYPES

1. What is it?

JavaScript is a high-level, interpreted (JIT-compiled) programming language used to make web pages interactive, and — via Node.js — to build servers too.

2. Why does it exist?

Browsers needed a lightweight scripting language to manipulate web pages on the fly, without reloading. It later grew into a full backend language (Node.js).

3. JavaScript vs Java

	JavaScript	Java
Type	Interpreted/JIT	Compiled to bytecode (JVM)
Typing	Dynamic	Static
Use	Web, servers (Node)	Enterprise apps, Android
Relation	Only shares the name	Completely different language

Interview trap: “JavaScript has nothing to do with Java.” Say this confidently if asked.

4. JS Engine & Interpreted/JIT

- Browsers run JS using an **engine** (V8 in Chrome/Node, SpiderMonkey in Firefox).
- Modern engines use **JIT (Just-In-Time) compilation**: code is parsed, an initial fast-but-unoptimized version runs immediately, and hot code paths get compiled to optimized machine code as they run.
- So JS is not purely “interpreted line by line” anymore — it’s a hybrid.

5. Variables & Constants

```
let age = 21;           // can be reassigned
const name = "Mahil";  // cannot be reassigned
var old = "legacy";    // old way, avoid in new code
```

6. Primitive Data Types

string, number, boolean, undefined, null, symbol, bigint

```
typeof "hello"    // "string"
typeof 42         // "number"
typeof true       // "boolean"
typeof undefined  // "undefined"
typeof null       // "object"    famous JS bug, memorize this
typeof Symbol()   // "symbol"
typeof 10n        // "bigint"
```

7. Reference Types

Objects, Arrays, Functions — stored by **reference**, not by value.

8. Primitive vs Reference — Comparison Table

Aspect	Primitive	Reference
Examples	string, number, boolean, null, undefined, symbol, bigint	object, array, function
Stored in	Stack (by value)	Heap (variable holds a reference/pointer)
Copying	Creates a real independent copy	Copies the reference — both point to same data
Comparison	Compared by value	Compared by reference (identity)
Mutability	Immutable	Mutable

```
let a = 10;
let b = a;
b = 20;
console.log(a); // 10 (unaffected)

let obj1 = { val: 10 };
let obj2 = obj1;
obj2.val = 20;
console.log(obj1.val); // 20 (both point to same object!)
```

9. null vs undefined

	null	undefined
Meaning	Intentional “no value”	Variable declared but not assigned
Set by typeof	Developer "object"	JavaScript automatically "undefined"

10. NaN

“Not a Number” — but its `typeof` is `"number"`. Result of invalid math operations.

```
console.log(0 / 0);           // NaN
console.log(typeof NaN);      // "number"
console.log(NaN === NaN);     // false (use Number.isNaN() to check)
```

11. Symbol & BigInt (brief — rarely core to placement interviews)

- `Symbol()` creates a guaranteed-unique value, used for hidden/unique object keys.
- `BigInt` (10n) represents integers larger than `Number.MAX_SAFE_INTEGER`.

12. Mutable vs Immutable

Primitives are immutable (you can't change a string in place, only create a new one). Objects/Arrays are mutable (you can change their contents without creating a new reference).

Interview Questions

1. What are JavaScript's primitive data types?
2. Why does `typeof null` return `"object"`?
3. What's the difference between `null` and `undefined`?
4. Is JavaScript compiled or interpreted?
5. What is NaN and how do you correctly check for it?

Interview Trap

`typeof null === "object"` is a long-standing JS bug kept for backward compatibility. Interviewers love asking this.

Quick Revision

- Primitives copy by value, references copy by reference.
 - `typeof null` → `"object"` (bug, memorize it).
 - Use `Number.isNaN()`, never `=== NaN`.
 - JS engines are JIT — not purely interpreted.
-

2. VARIABLES, SCOPE & HOISTING

1. What is it?

Scope defines **where** a variable is accessible. Hoisting is JavaScript's behavior of moving declarations to the top of their scope during compilation.

2. Why does it exist?

Scope prevents variables from leaking everywhere and colliding. Hoisting is a side-effect of how JS's two-phase (compile + execute) engine works.

3. Types of Scope

- **Global scope** — accessible everywhere.
- **Function scope** — `var` is function-scoped.
- **Block scope** — `let/const` are block-scoped (`{ }`).
- **Lexical scope** — a function can access variables from the scope in which it was *written* (not called).
- **Scope chain** — when a variable isn't found in the current scope, JS looks outward through parent scopes until it finds it or hits global scope.

```
function outer() {  
  let x = 10;  
  function inner() {  
    console.log(x); // lexical scope: inner "remembers" outer's x  
  }  
  inner();  
}
```

4. var vs let vs const

	var	let	const
Scope	Function	Block	Block
Reassignable	Yes	Yes	No
Redeclarable	Yes	No	No
Hoisted	Yes (initialized as undefined)	Yes (but in TDZ)	Yes (but in TDZ)

5. Hoisting — Output Question

```
console.log(a);  
var a = 10;  
// Output: undefined (NOT an error)
```

How it works: During the compile phase, JS “hoists” `var a` to the top of the scope and initializes it with `undefined`. The assignment (`a = 10`) happens later, in place. `console.log(a)` runs before the assignment, so it prints `undefined`.

```
console.log(b);  
let b = 10;  
// Output: ReferenceError: Cannot access 'b' before initialization
```

How it works: `let/const` are hoisted too, but they land in the **Temporal Dead Zone (TDZ)** — a period where the variable exists but cannot be accessed until its declaration line executes.

6. Temporal Dead Zone (TDZ)

The TDZ is the space between entering a scope and the actual `let/const` declaration line. Accessing the variable in this zone throws a `ReferenceError`.

Interview Definition

“Hoisting is JavaScript’s behavior of processing variable and function declarations before executing code, moving them to the top of their scope in memory.”

Interview Questions

1. What is hoisting?
2. Why does `var` print `undefined` but `let` throws an error when accessed before declaration?
3. What is the Temporal Dead Zone?
4. What is the scope chain?
5. Difference between function scope and block scope?

Interview Trap

Function declarations are hoisted **completely** (including their body), but function expressions (`const fn = function(){}`) are only hoisted as a variable (`undefined` until assignment).

```
sayHi(); // Works  
function sayHi(){ console.log("hi"); }  
  
sayBye(); // TypeError: sayBye is not a function  
var sayBye = function(){ console.log("bye"); };
```

Real Project Usage

Using `let/const` instead of `var` avoids accidental global leaks and bugs in loops (especially with closures inside loops — see Chapter 6).

Quick Revision

- Always prefer `const`, then `let`. Avoid `var`.
 - `var` → hoisted + `undefined`. `let/const` → hoisted but in TDZ.
 - Function declarations are fully hoisted; expressions are not.
 - Scope chain always looks outward, never inward.
-

3. OPERATORS & TYPE COERCION

1. What is it?

Type coercion is JavaScript automatically converting a value from one type to another (e.g., string to number) when needed for an operation.

2. Why does it exist?

JS is loosely typed — it tries to “make operations work” instead of throwing errors, which is convenient but leads to confusing behavior.

3. Implicit vs Explicit Coercion

```
// Implicit
"5" + 1    // "51" (number converted to string)
"5" - 1    // 4   (string converted to number)

// Explicit
Number("5")    // 5
String(5)      // "5"
Boolean(0)     // false
```

4. == vs ===

	== (loose)	=== (strict)
Checks	Value only (coerces types)	Value AND type
Recommended	Avoid	Always prefer

```
0 == "0"    // true  (coercion)
0 === "0"   // false
null == undefined // true
null === undefined // false
```

5. Truthy / Falsy Values

Falsy: false, 0, "", null, undefined, NaN **Everything else is truthy**, including "0", [], {} (these are objects/non-empty strings, so truthy!).

```
if ([]) console.log("runs"); // runs! empty array is truthy
if ("0") console.log("runs"); // runs! non-empty string is truthy
```

6. “JavaScript Weirdness” — Interview Favorites

```
console.log(1 + "1"); // "11"
console.log(1 - "1"); // 0
console.log("5" + 3); // "53"
console.log("5" - 3); // 2
console.log([] + []); // ""
console.log([] + {}); // "[object Object]"
console.log(true + true); // 2
console.log([1,2] + [3,4]); // "1,23,4"
console.log(NaN === NaN); // false
```

Interview Definition

“Type coercion is the automatic or implicit conversion of values from one data type to another, e.g., during comparisons or arithmetic operations.”

Interview Questions

1. Difference between `==` and `===`?
2. What are falsy values in JS?
3. Why does `[] + []` give `""`?
4. Why should you avoid `==`?
5. What does `"5" + 3` output and why?

Interview Trap

`typeof NaN` is `"number"`, and comparing `NaN === NaN` is always `false`.

Real Project Usage

Using `===` everywhere avoids subtle bugs when comparing API response values (which often arrive as strings) against numbers.

Quick Revision

- Always use `===/!==`.
 - Memorize the falsy list.
 - `+` on strings concatenates; other operators coerce to numbers.
 - `NaN !== NaN` — use `Number.isNaN()`.
-

4. FUNCTIONS

1. What is it?

A function is a reusable block of code that performs a task and can optionally return a value.

2. Why does it exist?

To avoid repeating code (DRY principle) and to organize logic into named, reusable units.

3. Types of Functions

```
// Function Declaration (hoisted fully)
function add(a, b) { return a + b; }

// Function Expression (not hoisted with body)
const sub = function(a, b) { return a - b; };

// Arrow Function (shorter syntax, no own `this`)
const mul = (a, b) => a * b;

// Anonymous function
setTimeout(function() { console.log("done"); }, 1000);

// Callback function - a function passed as an argument
function greet(name, callback) { callback(name); }
greet("Mahil", (n) => console.log(`Hi ${n}`));

// Higher-Order Function - takes/returns a function
function higherOrder(fn) { return fn(5); }
```

4. Default & Rest Parameters

```
function greet(name = "Guest") { console.log(`Hello ${name}`); }
greet(); // Hello Guest

function sum(...nums) { return nums.reduce((a,b) => a+b, 0); }
sum(1,2,3); // 6
```

5. Why are functions “first-class citizens”?

Because in JS, functions are treated like any other value: they can be - stored in variables - passed as arguments - returned from other functions - stored in arrays/objects

This is what enables callbacks, higher-order functions, and the entire functional style used in React (`.map()`, `.filter()`) and Node (`app.get(path, callback)`).

Interview Definition

“A function is first-class if it can be treated like any other value — assigned, passed, and returned.”

Interview Questions

1. Difference between function declaration and function expression?
2. What is a higher-order function? Give an example.
3. Why are arrow functions not suitable as object methods?
4. What are default parameters used for?
5. What is a callback function?

Interview Trap

Arrow functions **don’t have their own this** — a huge trap covered fully in Chapter 5.

Real Project Usage

- Callbacks → event handlers, `.then()`
- Higher-order functions → `array.map()`, `array.filter()` used constantly in React rendering.

Quick Revision

- Function declarations are hoisted; expressions/arrows are not.
 - Default params avoid manual `undefined` checks.
 - Rest params (`...args`) collect remaining arguments into an array.
 - “First-class” = functions behave like values.
-

5. this KEYWORD

1. What is it?

`this` refers to the object that is currently “executing” the function — but its value depends entirely on **how the function is called**, not where it’s defined (except arrow functions).

2. Why does it exist?

So the same function/method can operate on different objects dynamically.

3. THIS — Interview Cheat Sheet

Context	Value of <code>this</code>
Global scope (non-strict)	<code>window</code> (browser) / <code>global</code> (Node)
Regular function call	<code>undefined</code> (strict mode) or global object
Object method call	The object before the dot
Arrow function	Inherits <code>this</code> from the enclosing (lexical) scope
Constructor function / class	The newly created instance
<code>call()/apply()</code>	Explicitly whatever object you pass
<code>bind()</code>	Permanently bound to whatever object you pass

4. Examples

```
const obj = {
  name: "Mahil",
  greet() { console.log(this.name); }
};
obj.greet(); // "Mahil" - this = obj
```

```
function standalone() { console.log(this); }
standalone(); // undefined (strict mode) - no object before the dot
```

```
const arrowObj = {
  name: "Mahil",
  greet: () => console.log(this.name)
};
arrowObj.greet(); // undefined - arrow has NO own `this`, uses outer (module/global) scope
```

5. call, apply, bind

```
function intro() { console.log(`I am ${this.name}`); }
const person = { name: "Mahil" };

intro.call(person);           // calls immediately, args passed individually
intro.apply(person, []);      // calls immediately, args passed as array
const boundFn = intro.bind(person); // returns a NEW function, call later
boundFn();
```

Output Question

```
const obj = {
  name: "A",
  regular: function() { console.log(this.name); },
  arrow: () => { console.log(this.name); }
};
obj.regular(); // "A"
obj.arrow();    // undefined
```

Interview Definition

“this is determined by the call-site of a function — who is calling it — except in arrow functions, where this is lexically inherited.”

Interview Questions

1. What determines the value of **this** in JS?
2. Why don't arrow functions have their own **this**?
3. Difference between **call()**, **apply()**, and **bind()**?
4. What is **this** inside a regular function called standalone?
5. What is **this** inside a class method?

Interview Trap

Using a regular function as a callback often loses the intended **this** (e.g., inside **setTimeout** or event handlers) — arrow functions are commonly used to fix this.

Real Project Usage

In React class components (older style), **this.handleClick = this.handleClick.bind(this)** was required in the constructor precisely because of this behavior. In Node.js Express middleware, **this** inside regular callback functions usually doesn't refer to what you'd expect.

Quick Revision

- **this** = “who called me”, except arrows = “what surrounds me”.

- `bind()` returns a new function; `call()`/`apply()` invoke immediately.
 - Object method shorthand (`greet() {}`) gets normal `this` behavior.
 - Arrow functions used as object methods are a common trap.
-

6. CLOSURES

1. What is it?

A closure is a function that “remembers” the variables from the scope it was created in, even after that outer scope has finished executing.

2. Why does it exist?

It’s a direct result of lexical scoping — but it’s incredibly useful for creating private data and function factories.

Think of it like this: the inner function carries a backpack containing all the variables it needs from where it was born, even after it leaves home.

3. Basic Example — Counter

```
function createCounter() {  
  let count = 0;           // private variable  
  return function () {  
    count++;  
    return count;  
  };  
}
```

```
const counter = createCounter();  
console.log(counter()); // 1  
console.log(counter()); // 2  
console.log(counter()); // 3
```

How it works: `createCounter()` runs and finishes, but the returned inner function keeps a reference to `count` (its lexical environment), so `count` is never garbage-collected and keeps incrementing across calls.

4. Function Factories

```
function multiplier(x) {  
  return function(y) { return x * y; };  
}
```

```
const double = multiplier(2);
console.log(double(5)); // 10
```

5. Closures inside Loops — Classic Trap

```
for (var i = 1; i <= 3; i++) {
  setTimeout(() => console.log(i), 1000);
}
// Output: 4, 4, 4 (NOT 1, 2, 3)
```

Why: `var` is function-scoped, so all three callbacks share the *same* `i`. By the time `setTimeout` fires, the loop has finished and `i` is 4.

Fix: use `let` (block-scoped — a new `i` per iteration):

```
for (let i = 1; i <= 3; i++) {
  setTimeout(() => console.log(i), 1000);
}
// Output: 1, 2, 3
```

Interview Definition

“A closure is the combination of a function and the lexical environment within which that function was declared, allowing it to access outer variables even after the outer function has returned.”

Interview Questions

1. What is a closure? Give a real example.
2. Why does the classic `var` + `setTimeout` loop print the same number multiple times?
3. How do closures enable private variables in JavaScript?
4. What is a function factory?
5. How would you implement a counter using closures?

Interview Trap

Confusing closures with just “nested functions” — the key part is that the variables **persist** after the outer function returns.

Real Project Usage

- React custom hooks rely heavily on closures (state capturing).
- Debounce/throttle utility functions use closures to persist a timer variable across calls.
- Module patterns (before ES6 modules) used closures to create private state.

Quick Revision

- Closures = function + its remembered lexical scope.
- Used for private variables, factories, callbacks.

- `var` in loops + async callbacks = classic bug; fix with `let`.
 - Every React custom hook you write is basically a closure.
-

7. ARRAYS & ARRAY METHODS

1. What is it?

An array is an ordered, index-based collection of values.

2. Creating & Basic Operations

```
const arr = [1, 2, 3];
arr[0];           // 1 (indexing)
arr.push(4);      // add to end -> [1,2,3,4]
arr.pop();        // remove from end
arr.unshift(0);   // add to start
arr.shift();      // remove from start
arr.length;       // 4
for (const item of arr) console.log(item); // iteration
```

3. Array Methods — Full Table

Method	Purpose	Returns	Mutates Original?
map()	Transform each element	New array	No
filter()	Keep elements matching condition	New array	No
reduce()	Reduce to single value	Any value	No
forEach()	Run side-effect per element	undefined	No
find()	First matching element	Element or undefined	No
findIndex()	Index of first match	Number (-1 if none)	No
some()	At least one matches?	Boolean	No
every()	All match?	Boolean	No
includes()	Value exists?	Boolean	No
sort()	Sort elements	Same array (sorted)	Yes
reverse()	Reverse order	Same array	Yes
slice(a,b)	Extract portion	New array	No
splice(a,b,...)	Insert/remove elements	Removed elements	Yes
concat()	Merge arrays	New array	No
flat()	Flatten nested arrays	New array	No

4. Examples

```
const nums = [1, 2, 3, 4, 5];

nums.map(n => n * 2);           // [2,4,6,8,10]
nums.filter(n => n % 2 === 0); // [2,4]
nums.reduce((sum, n) => sum + n, 0); // 15
nums.find(n => n > 3);           // 4
nums.some(n => n > 4);           // true
nums.every(n => n > 0);          // true
nums.includes(3);               // true

nums.slice(1, 3); // [2,3] - does NOT mutate
nums.splice(1, 2); // removes [2,3] from nums - MUTATES nums!
```

Interview Questions

1. Difference between `map()` and `forEach()`?
2. Difference between `slice()` and `splice()`?
3. How does `reduce()` work internally?
4. Which array methods mutate the original array?
5. How do you flatten a nested array?

Interview Trap

`map()` vs `forEach()`: `map()` returns a new array, `forEach()` returns `undefined`. Using `forEach()` when you meant `map()` is a very common beginner (and interview-trap) mistake. `slice()` vs `splice()`: `slice` is **non-mutating** (extraction), `splice` is **mutating** (insertion/removal) — sound similar, behave oppositely.

Real Project Usage

`map()` is used constantly in React to render lists of components from arrays of data. `filter()` is used to remove items (e.g., delete a todo). `reduce()` is used to calculate totals (e.g., cart price).

Quick Revision

- `map/filter` → new array, non-mutating, used everywhere in React.
 - `reduce` → single accumulated value.
 - `sort, reverse, splice` → mutate the original array (be careful with state in React!).
 - `slice` → safe copy of a portion.
-

8. OBJECTS & OBJECT METHODS

1. What is it?

An object is a collection of key-value pairs used to represent structured data.

2. Creating & Accessing

```
const user = {  
  name: "Mahil",  
  age: 21,  
  ["skill" + 1]: "JS"  // computed property name  
};  
  
user.name;      // dot notation  
user["age"];    // bracket notation (needed for dynamic keys)  
user.city = "Bhopal";  // add property  
delete user.age;    // remove property
```

3. Nested Objects

```
const student = {  
  name: "Mahil",  
  address: { city: "Bhopal", state: "MP" }  
};  
student.address.city; // "Bhopal"
```

4. Object Methods

```
const obj = { a: 1, b: 2 };  
  
Object.keys(obj);    // ["a", "b"]  
Object.values(obj);  // [1, 2]  
Object.entries(obj); // [["a", 1], ["b", 2]]  
Object.assign({}, obj, { c: 3 }); // {a:1, b:2, c:3} - shallow merge
```

5. Object Destructuring (preview — full detail in Ch. 9)

```
const { name, age } = user;
```

Interview Questions

1. Difference between `Object.keys()` and `Object.entries()`?
2. What does `Object.assign()` do, and is it a deep or shallow copy?
3. How do you loop over an object's properties?
4. What are computed property names?
5. How do you check if a property exists on an object?

Interview Trap

`Object.assign()` and the spread operator (`{...obj}`) both do **shallow** copies only — nested objects are still shared by reference.

```
const a = { info: { x: 1 } };
const b = { ...a };
b.info.x = 99;
console.log(a.info.x); // 99 - nested object was shared!
```

Real Project Usage

In React, updating state immutably is done with spread: `setUser({ ...user, age: 22 })`. In Node/Express, request bodies arrive as objects that you destructure: `const { name, email } = req.body`.

Quick Revision

- Dot notation for known keys, bracket notation for dynamic keys.
 - `Object.keys/values/entries` → array conversions for iteration.
 - `Object.assign/spread` = shallow copy only.
 - Deep copy needs `structuredClone()` or `JSON.parse(JSON.stringify(obj))` (loses functions/undefined).
-

9. DESTRUCTURING, SPREAD & REST

1. What is it?

- **Destructuring**: unpacking values from arrays/objects into variables.
- **Spread (...)**: expands an iterable into individual elements.
- **Rest (...)**: collects multiple elements into a single array/object.

2. Spread vs Rest — Key Difference

	Spread	Rest
Direction	Expands a collection into individual items	Collects individual items into a collection
Used in	Function calls, array/object literals	Function parameters, destructuring

```
// SPREAD - expanding
const arr1 = [1, 2, 3];
const arr2 = [...arr1, 4, 5]; // [1,2,3,4,5]
Math.max(...arr1);           // spreads array as separate args

const obj1 = { a: 1 };
const obj2 = { ...obj1, b: 2 }; // {a:1, b:2}

// REST - collecting
function sum(...nums) {      // nums = [args collected as array]
  return nums.reduce((a,b)=>a+b);
}

const [first, ...rest] = [1, 2, 3, 4]; // first=1, rest=[2,3,4]
```

3. Array & Object Destructuring

```
const [a, b] = [10, 20];
const { name, age } = { name: "Mahil", age: 21 };
const { name: fullName } = { name: "Mahil" }; // rename
```

```

const { city = "Unknown" } = {};           // default value

function greet({ name, age }) {           // destructure params directly
  console.log(`${name} is ${age}`);
}

```

Interview Questions

1. Difference between spread and rest operators (they use the same ... syntax)?
2. How do you swap two variables using destructuring?
3. How do you skip elements while destructuring an array?
4. What happens if you spread an object with nested objects — deep or shallow copy?
5. How do you set default values while destructuring?

Interview Trap

Spread creates only a **shallow copy** — same nested-reference issue as `Object.assign()` (see Chapter 8). Also: ... looks identical for spread and rest; the difference is purely based on **position** (left side of `=` → rest, right side / literal → spread).

Real Project Usage

Destructuring `req.body` in Express (`const { email, password } = req.body`), destructuring props in React (`function Card({ title, image })`), spreading state updates immutably (`setState({ ...state, loading: true })`).

Quick Revision

- Spread: expand. Rest: collect. Same symbol, opposite jobs, told apart by position.
 - Destructuring supports renaming and default values.
 - Spread copies are shallow — nested objects still shared.
 - React and Express code use this pattern constantly.
-

10. STRINGS & STRING METHODS

1. What is it?

A string is a sequence of characters, immutable in JS (any “modification” creates a new string).

2. Common Methods

```
const s = "Hello World";

s.length;           // 11
s.toUpperCase();     // "HELLO WORLD"
s.toLowerCase();     // "hello world"
s.includes("World"); // true
s.indexOf("World");  // 6
s.slice(0, 5);       // "Hello"
s.split(" ");        // ["Hello", "World"]
s.trim();            // removes surrounding whitespace
s.replace("World", "JS"); // "Hello JS"
s.charAt(0);         // "H"
`Value: ${s}`;       // template literal - string interpolation
```

Interview Trap

Strings are **immutable**: `s[0] = "Y"` does NOT change the string — it silently does nothing (no error, no effect).

Real Project Usage

Template literals are used everywhere for building dynamic strings (URLs, JSX text, error messages).

Quick Revision

- Strings are immutable primitives.
- `slice()` doesn't mutate; template literals replace old string concatenation.
- `split()/join()` convert between strings and arrays.

11. DOM & BROWSER JAVASCRIPT

1. What is it?

The DOM (Document Object Model) is a tree-like representation of an HTML page that JavaScript can read and modify.

2. Common Operations

```
document.getElementById("title");
document.querySelector(".card");
document.querySelectorAll("li");

el.textContent = "New text";
el.innerHTML = "<b>Bold</b>";
el.style.color = "red";

const div = document.createElement("div");
document.body.appendChild(div);
el.remove();

el.addEventListener("click", () => console.log("clicked"));
```

Interview Questions

1. Difference between `textContent` and `innerHTML`?
2. Difference between `querySelector` and `getElementById`?
3. Why is direct DOM manipulation avoided in React?

Interview Trap

`innerHTML` can expose you to XSS (cross-site scripting) if you insert unsanitized user input — prefer `textContent` for plain text.

Real Project Usage

In plain JS/vanilla projects, DOM manipulation is direct. In React, you almost never touch the DOM directly — React's virtual DOM handles it, which is why understanding DOM basics still matters (React ultimately produces real DOM operations under the hood).

Quick Revision

- DOM = tree structure of the page.
 - `textContent` = safe text. `innerHTML` = can inject HTML (risk of XSS).
 - React abstracts this away via the virtual DOM.
-

12. EVENTS & EVENT HANDLING

1. What is it?

An event is an action (click, keypress, submit). An event listener is a function that runs when that event happens.

2. Event Bubbling vs Capturing

- **Bubbling** (default): event fires on the target, then bubbles UP through its ancestors.
- **Capturing**: event fires on ancestors first, going DOWN to the target (opt-in via `{ capture: true }`).

3. Event Delegation

Instead of attaching a listener to every child element, attach ONE listener to a parent and use `event.target` to detect which child was actually clicked.

```
document.getElementById("list").addEventListener("click", (e) => {  
  if (e.target.tagName === "LI") {  
    console.log("Clicked:", e.target.textContent);  
  }  
});
```

Why: Efficient for dynamic lists (new items added later still work without re-attaching listeners), and reduces memory usage.

4. preventDefault() vs stopPropagation()

```
form.addEventListener("submit", (e) => {  
  e.preventDefault();    // stops default browser action (e.g., page reload)  
});
```

```
child.addEventListener("click", (e) => {  
  e.stopPropagation();    // stops event from bubbling to parent  
});
```

Interview Questions

1. What is event delegation and why use it?

2. Difference between event bubbling and capturing?
3. Difference between `preventDefault()` and `stopPropagation()`?

Interview Trap

Confusing `preventDefault()` (stops the browser's default behavior) with `stopPropagation()` (stops the event from reaching parent elements) — these solve completely different problems.

Real Project Usage

Event delegation is exactly how libraries like React handle events internally (React attaches one listener at the root and uses synthetic events). In forms, `e.preventDefault()` is used constantly to stop page reloads on submit.

Quick Revision

- Bubbling = default, capturing = opt-in.
 - Delegation = one listener on parent, checks `e.target`.
 - `preventDefault` = stop default action. `stopPropagation` = stop bubbling.
-

13. CALLBACKS

1. What is it?

A callback is a function passed into another function, to be executed later (often after some operation completes).

2. Why does it exist?

JS is single-threaded — callbacks let it hand off “what to do next” without blocking, especially for async work (timers, network calls).

```
function fetchData(callback) {
  setTimeout(() => {
    callback("data loaded");
  }, 1000);
}
fetchData((result) => console.log(result));
```

3. Callback Hell

```
getUser(id, (user) => {
  getPosts(user.id, (posts) => {
    getComments(posts[0].id, (comments) => {
      console.log(comments); // deeply nested - "pyramid of doom"
    });
  });
});
```

This is exactly the problem Promises and async/await were built to solve (see next two chapters).

Interview Questions

1. What is a callback function?
2. What is “callback hell” and how do you avoid it?
3. Are all callbacks asynchronous? (No — `array.map(callback)` is synchronous.)

Interview Trap

Not all callbacks are async! `.map()`, `.filter()`, `.forEach()` callbacks run synchronously — only callbacks tied to timers, I/O, or promises are asynchronous.

Real Project Usage

Express route handlers are callbacks: `app.get("/users", (req, res) => {...})`. Node's older APIs (`fs.readFile(path, callback)`) use the “error-first callback” pattern.

Quick Revision

- Callback = function passed to run later.
 - Not all callbacks are async.
 - Callback hell → solved by Promises/async-await.
-

14. ASYNCHRONOUS JAVASCRIPT

1. What is it?

Asynchronous JS lets long-running operations (network calls, timers, file reads) happen in the background without freezing the rest of the program.

2. Why does it exist?

JavaScript runs on a **single thread**. Without async behavior, a slow operation (like a network request) would freeze the entire page/app.

3. The Progression

Synchronous JS → Asynchronous JS → Callbacks → Callback Hell → Promises → async/await → Event Loop

```
console.log("1");
setTimeout(() => console.log("2"), 0);
console.log("3");
// Output: 1, 3, 2
```

Why: `setTimeout` hands its callback to the browser/Node's Web API, and JS continues executing the rest of the synchronous code first. The callback only runs once the call stack is empty (explained fully in Chapter 17 — Event Loop).

Interview Definition

“Asynchronous JavaScript allows the engine to start a task and continue running other code before that task completes, using callbacks, promises, or async/await to handle the result later.”

Quick Revision

- JS is single-threaded but non-blocking via async patterns.
- `setTimeout(fn, 0)` still runs AFTER all synchronous code.
- This chapter is conceptual groundwork for Promises → async/await → Event Loop.

15. PROMISES

1. What is it?

A Promise is an object representing the eventual completion (or failure) of an asynchronous operation.

2. Why does it exist?

To fix callback hell by giving async code a flat, chainable structure instead of deep nesting.

3. Promise States

- **Pending** — initial state, neither fulfilled nor rejected.
- **Fulfilled** — operation completed successfully (**resolve** was called).
- **Rejected** — operation failed (**reject** was called).

A promise settles (fulfilled/rejected) exactly once and is then immutable.

4. Basic Syntax

```
const promise = new Promise((resolve, reject) => {
  const success = true;
  setTimeout(() => {
    if (success) resolve("Data loaded");
    else reject("Error occurred");
  }, 1000);
});
```

```
promise
  .then((result) => console.log(result))
  .catch((err) => console.log(err))
  .finally(() => console.log("Done, either way"));
```

5. Fixing Callback Hell with Promise Chaining

```
getUser(id)
  .then(user => getPosts(user.id))
  .then(posts => getComments(posts[0].id))
```

```
.then(comments => console.log(comments))
.catch(err => console.error(err));
```

6. Promise Combinators

Method	Behavior
<code>Promise.all()</code>	Waits for ALL to resolve; rejects immediately if ANY rejects
<code>Promise.allSettled()</code>	Waits for ALL, returns status of each (never short-circuits)
<code>Promise.race()</code>	Resolves/rejects as soon as the FIRST settles (win or lose)
<code>Promise.any()</code>	Resolves as soon as the FIRST fulfills; rejects only if ALL reject

```
Promise.all([p1, p2, p3])
  .then(results => console.log(results)) // [r1, r2, r3]
  .catch(err => console.log("one failed:", err));
```

Interview Definition

“A Promise is an object representing a value that may not be available yet, but will be resolved (or rejected) at some point in the future.”

Interview Questions

1. What are the three states of a Promise?
2. Difference between `Promise.all()` and `Promise.allSettled()`?
3. What happens if one promise rejects inside `Promise.all()`?
4. How do you handle errors in a promise chain?
5. Difference between `Promise.race()` and `Promise.any()`?

Interview Trap

`Promise.all()` fails fast — if even one promise rejects, the whole thing rejects immediately, even if others would have succeeded. Use `Promise.allSettled()` if you need results from all regardless of failure.

Real Project Usage

`fetch()` returns a Promise. `Promise.all()` is used to fetch multiple API endpoints in parallel (e.g., user data + posts data at once) instead of one after another.

Quick Revision

- 3 states: pending → fulfilled/rejected (one-way, permanent).

- `.then/.catch/.finally` chain, replacing nested callbacks.
 - `all` = fail-fast, `allSettled` = always waits for everyone, `race/any` = first to settle/fulfill.
-

16. ASYNC/AWAIT

1. What is it?

async/await is syntactic sugar over Promises that lets asynchronous code read like synchronous code.

2. Basic Syntax

```
async function getUserData() {  
  try {  
    const user = await fetch("/api/user");  
    const data = await user.json();  
    console.log(data);  
  } catch (err) {  
    console.log("Error:", err);  
  }  
}
```

3. How it works

- async before a function means it **always returns a Promise**.
- await pauses execution of THAT function (not the whole program) until the awaited promise settles.
- Errors are caught using regular try/catch instead of .catch().

4. Sequential vs Parallel Await (common interview trap)

```
// SLOW - sequential, waits for each one by one  
const a = await fetchA(); // waits  
const b = await fetchB(); // then waits again  
  
// FAST - parallel  
const [a, b] = await Promise.all([fetchA(), fetchB()]);
```

Interview Definition

“async/await is syntax built on top of Promises that allows asynchronous code to be written and read in a synchronous style, improving readability.”

Interview Questions

1. What does the `async` keyword do to a function's return value?
2. How do you handle errors with `async/await`?
3. What's the performance difference between awaiting sequentially vs using `Promise.all()`?
4. Can you use `await` outside an `async` function? (Only at the top level of ES modules.)
5. Is `async/await` synchronous or asynchronous under the hood?

Interview Trap

`async/await` does NOT make code synchronous — it's still Promise-based and non-blocking; it just *looks* synchronous.

Real Project Usage

Almost all Express route handlers and React data-fetching code (in `useEffect`) use `async/await` for clean API calls.

Quick Revision

- `async` function → always returns a Promise.
 - `await` → pauses only that function, not the whole thread.
 - Use `Promise.all()` when operations don't depend on each other, to run in parallel.
 - Errors handled via `try/catch`.
-

17. EVENT LOOP (Most important async topic for interviews)

1. What is it?

The Event Loop is the mechanism that allows JavaScript (single-threaded) to handle asynchronous operations by coordinating the Call Stack, Web APIs, and the Task Queues.

2. The Components

Call Stack <- synchronous code executes here, one frame at a time

(when async op is called, e.g. setTimeout/fetch)

Web APIs / Node <- timers, network calls, DOM events run OUTSIDE the JS thread
runtime APIs

(once done, callback is queued)

Macrotask Queue	Microtask Queue
(setTimeout, setInterval, I/O, UI rendering)	(Promise.then/catch, queueMicrotask, async/await resume)

EVENT LOOP

(checks: is Call Stack empty? if so, run ALL microtasks first,
THEN take ONE macrotask, then repeat)

3. The Golden Rule

The Event Loop only moves a task from a queue to the Call Stack when the Call Stack is completely empty. ALL microtasks are drained completely before the next macrotask runs.

Priority order: Synchronous code > Microtasks (Promises) > Macrotasks (setTimeout, setInterval, I/O)

4. Classic Output Question — Walkthrough

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

Promise.resolve().then(() => {
  console.log("C");
});

console.log("D");

// Output: A, D, C, B
```

Step-by-step: 1. `console.log("A")` → synchronous → prints immediately → A 2. `setTimeout(...)` → handed off to Web API, callback queued in **macrotask** queue (even with 0ms delay). 3. `Promise.resolve().then(...)` → callback queued in **microtask** queue. 4. `console.log("D")` → synchronous → prints immediately → D 5. Call stack is now empty. Event loop checks microtask queue FIRST → runs C. 6. Microtask queue is empty. Event loop now takes ONE task from macrotask queue → runs B. 7. Final output: A D C B

5. More Output Practice

```
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
Promise.resolve().then(() => console.log("4"));
setTimeout(() => console.log("5"), 0);
console.log("6");
// Output: 1, 6, 3, 4, 2, 5
// (all sync first, then ALL microtasks in order, then macrotasks in order)

async function test() {
  console.log("start");
  await null;           // <- yields control, rest of function becomes a microtask
  console.log("end");
}
test();
console.log("outside");
// Output: start, outside, end
```

Interview Definition

“The Event Loop continuously checks whether the call stack is empty, and if so, pulls the next task from the microtask queue (fully draining it) before pulling a single task from the macrotask queue.”

Interview Questions

1. What is the Event Loop?
2. Why does `setTimeout(fn, 0)` not run immediately?
3. What runs first — a `Promise.then()` or a `setTimeout()` callback?
4. What is the Call Stack?
5. Why are microtasks always fully drained before the next macrotask?

Interview Trap

People assume `setTimeout(fn, 0)` runs “immediately” — it doesn’t. It’s still a macrotask and always runs after ALL synchronous code AND all pending microtasks, no matter how small the delay.

Real Project Usage

Understanding the event loop explains why UI doesn’t freeze during API calls in React, and why Node.js can handle thousands of concurrent connections on a single thread (non-blocking I/O).

Quick Revision

- Order: sync code → all microtasks → one macrotask → repeat.
 - Microtasks: Promises, `queueMicrotask`, `async/await` continuations.
 - Macrotasks: `setTimeout`, `setInterval`, I/O, UI rendering.
 - `setTimeout(fn, 0)` immediate — it’s still last in this priority order.
-

18. MICROTASKS & MACROTASKS

1. What is it?

Two separate queues the Event Loop uses, with **microtasks always taking priority** over macrotasks.

2. Table

	Microtask	Macrotask
Examples	<code>Promise.then/catch/finally</code> , <code>queueMicrotask()</code> , <code>async/await</code> resume	<code>setTimeout</code> , <code>setInterval</code> , <code>setImmediate</code> (Node), I/O, UI events
Priority	Higher — ALL drained before next macrotask	Lower — one runs per event loop tick

Interview Trap

If you keep adding microtasks recursively inside a `.then()`, macrotasks (like `setTimeout`) can be **starved** (delayed indefinitely) because microtasks always run first and the queue never empties.

Quick Revision

- Microtasks > Macrotasks, always.
- This distinction is what makes many event-loop output questions solvable.

19. ERROR HANDLING

1. What is it?

Techniques for gracefully catching and responding to runtime errors instead of crashing the program.

2. try/catch/finally

```
try {  
    JSON.parse("invalid json");  
} catch (err) {  
    console.log("Caught:", err.message);  
} finally {  
    console.log("Always runs");  
}
```

3. Handling Async Errors

```
// Promises  
fetchData().catch(err => console.log(err));  
  
// async/await  
async function load() {  
    try {  
        await fetchData();  
    } catch (err) {  
        console.log(err);  
    }  
}
```

4. Custom Errors

```
class ValidationError extends Error {  
    constructor(message) {  
        super(message);  
        this.name = "ValidationError";  
    }  
}  
  
throw new ValidationError("Invalid input");
```

Interview Questions

1. Why does `try/catch` alone not catch errors inside a `.then()` promise chain?
2. How do you create a custom error class?
3. Does `finally` run even if there's a `return` inside `try`? (Yes.)

Interview Trap

`try/catch` does NOT catch errors from asynchronous callbacks that run later (e.g. inside a raw `setTimeout`) unless you're specifically inside an `async` function using `await`.

```
try {  
  setTimeout(() => { throw new Error("oops"); }, 100);  
} catch (e) {  
  // NEVER catches this - the throw happens after try/catch already finished  
}
```

Real Project Usage

Express uses centralized error-handling middleware: `app.use((err, req, res, next) => {...})`.
React uses Error Boundaries for rendering errors.

Quick Revision

- `try/catch` works for sync code and `awaited` promises — not raw `async` callbacks.
 - Always `.catch()` or wrap `await`s in `try/catch` for API calls.
 - Custom error classes extend `Error` for meaningful error types.
-

20. ES6+ FEATURES (Quick Reference)

Most of these are already covered in depth earlier — this is a consolidated recap.

Feature	Example
<code>let/const</code>	Block scoping (Ch. 2)
Arrow functions	<code>(a,b) => a+b</code> (Ch. 4)
Template literals	<code>`Hi \${name}`</code>
Destructuring	<code>const {a,b} = obj</code> (Ch. 9)
Spread/Rest	<code>...arr</code> (Ch. 9)
Default parameters	<code>function f(x=1){}</code> (Ch. 4)
Classes	<code>class Person {}</code> (Ch. 23)
Modules	<code>import/export</code> (Ch. 21)
Promises / <code>async-await</code>	Ch. 15–16
Optional chaining <code>?.</code>	<code>user?.address?.city</code> — avoids error if <code>address</code> is undefined
Nullish coalescing <code>??</code>	<code>const val = input ?? "default"</code> — only falls back for <code>null/undefined</code> , NOT other falsy values
<code>for...of</code>	Iterates over values (arrays, strings, Maps, Sets)
<code>for...in</code>	Iterates over keys (objects — also array indices, avoid for arrays)
Map	Key-value store, ANY type as key, preserves insertion order
Set	Collection of unique values

Optional Chaining vs Nullish Coalescing — Trap

```
const count = 0;
console.log(count || 10); // 10 - WRONG if 0 is a valid value! (|| treats 0 as falsy)
console.log(count ?? 10); // 0 - CORRECT, ?? only checks null/undefined
```

`for...of` vs `for...in`

```
const arr = ["a", "b", "c"];
for (const val of arr) console.log(val); // "a","b","c" (values)
for (const key in arr) console.log(key); // "0","1","2" (indices/keys, as strings!)
```

Map vs Object

	Map	Object
Key types	Any type (objects, functions, etc.)	Strings/Symbols only
Order	Guaranteed insertion order	Mostly insertion order, but not guaranteed for all key types
Size	<code>.size</code> property	Manual (<code>Object.keys(obj).length</code>)
Iterable	Directly iterable	Needs <code>Object.entries()</code>

```
const m = new Map();
m.set("a", 1);
m.set("b", 2);
m.get("a");    // 1
m.size;        // 2

const s = new Set([1, 2, 2, 3]);
console.log(s); // Set(3) {1, 2, 3} - duplicates removed automatically
```

Interview Questions

1. Difference between `?.` and `???`
2. Why is `??` safer than `||` for default values?
3. When would you use a `Map` instead of a plain object?
4. Difference between `for...of` and `for...in`?
5. How does `Set` help remove duplicates from an array? (`[...new Set(arr)]`)

Interview Trap

`array || 10` treats `0`, `""`, `false` as “missing” — use `??` when only `null/undefined` should trigger the fallback.

Real Project Usage

Optional chaining is used constantly with API responses (`response?.data?.user?.name`) to avoid crashes when a field is missing. `Set` is a quick one-liner for deduplication in data-processing logic.

Quick Revision

- `??` only falls back on `null/undefined`; `||` falls back on ANY falsy value.
- `for...of` = values, `for...in` = keys (avoid on arrays).
- `Map` > object when keys aren't simple strings or order/size matters.

21. MODULES

1. What is it?

Modules let you split code across multiple files, exporting/importing specific pieces instead of writing everything globally.

2. ES Modules (used in React/modern JS)

```
// math.js
export const add = (a, b) => a + b;
export default function multiply(a, b) { return a * b; }

// app.js
import multiply, { add } from "./math.js";
```

3. CommonJS (used traditionally in Node.js)

```
// math.js
module.exports = { add, multiply };

// app.js
const { add, multiply } = require("./math.js");
```

Interview Questions

1. Difference between ES Modules and CommonJS?
2. Difference between named export and default export?
3. Can you have multiple default exports in a file? (No — only one.)

Interview Trap

CommonJS (`require`) is synchronous and loads at runtime; ES Modules (`import`) are statically analyzed at compile time (allowing tree-shaking/dead-code elimination in bundlers like Webpack/Vite).

Real Project Usage

React/Vite projects use ES Modules (`import/export`) throughout. Plain Node.js backend scripts traditionally use CommonJS (`require`), though modern Node also supports ES Modules with `"type": "module"` in `package.json`.

Quick Revision

- Named exports: many per file, imported with `{ }`.
 - Default export: one per file, imported without `{ }`.
 - CommonJS = Node's traditional system; ES Modules = the modern universal standard.
-

22. PROTOTYPES & PROTOTYPE CHAIN

1. What is it?

Every JavaScript object has an internal link to another object called its **prototype**, from which it can inherit properties and methods. This chain of links is the **prototype chain**.

2. Why does it exist?

JavaScript doesn't have "classical" class-based inheritance at its core — it uses **prototypal inheritance**, where objects inherit directly from other objects.

3. prototype vs __proto__

	prototype	__proto__
Exists on	Functions (specifically constructor functions)	Every object instance
Purpose	The blueprint object that will become the <code>__proto__</code> of instances created via <code>new</code>	The actual link to the object's parent in the prototype chain
Example	<code>Person.prototype.greet = ...</code>	<code>mahil.__proto__ === Person.prototype</code>

Think of it like this: **prototype** is the "recipe" a constructor function keeps for making objects, and **__proto__** is the actual "family link" every created object carries back to that recipe.

```
function Person(name) {  
  this.name = name;  
}  
Person.prototype.greet = function () {  
  console.log(`Hi, I'm ${this.name}`);  
};
```

```
const mahil = new Person("Mahil");  
mahil.greet(); // "Hi, I'm Mahil" - found via prototype chain, not on `mahil` itself
```

```
console.log(mahil.__proto__ === Person.prototype); // true
```

4. Prototype Chain Diagram

```
mahil ---> Person.prototype ---> Object.prototype ---> null
(instance)   (has greet())       (has toString(), etc.)
```

When you access `mahil.greet()`, JS first looks on `mahil` itself, doesn't find it, walks UP to `Person.prototype`, finds `greet` there.

5. Prototypal Inheritance

```
function Student(name, course) {
  Person.call(this, name);    // borrow Person's constructor logic
  this.course = course;
}
Student.prototype = Object.create(Person.prototype); // link prototypes
Student.prototype.constructor = Student;
```

Interview Definition

“Every object has a hidden link (`__proto__`) to another object (its prototype). When a property isn't found on the object itself, JS looks up this chain until it finds it or reaches `null`.”

Interview Questions

1. What is the difference between `prototype` and `__proto__`?
2. What is the prototype chain?
3. How does JS achieve inheritance without classes (under the hood)?
4. What does `Object.create()` do?
5. What happens if a property isn't found anywhere in the prototype chain?

Interview Trap

class syntax in JS (Chapter 23) is NOT a new inheritance model — it's **syntactic sugar over prototypal inheritance**. Under the hood, `class` still uses `prototype` and `__proto__`.

Real Project Usage

Understanding this explains why array methods like `.map()` exist on every array (they live on `Array.prototype`), and why adding a method to `Array.prototype` would make it available on ALL arrays (generally discouraged in production).

Quick Revision

- `prototype` = blueprint on constructor functions.

- `__proto__` = actual link on instances, points to that blueprint.
 - Prototype chain ends at `null`.
 - `class` in JS = sugar over this same system.
-

23. CLASSES & OOP IN JAVASCRIPT

1. What is it?

class syntax (ES6) provides a cleaner way to write constructor-function + prototype patterns for object-oriented programming.

2. Basic Syntax

```
class Person {  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
  greet() { // goes on Person.prototype automatically  
    console.log(`Hi, I'm ${this.name}`);  
  }  
}  
  
const mahil = new Person("Mahil", 21);  
mahil.greet();
```

3. Inheritance — extends & super

```
class Student extends Person {  
  constructor(name, age, course) {  
    super(name, age); // must call before using `this`  
    this.course = course;  
  }  
  greet() {  
    super.greet(); // call parent's method too  
    console.log(`Studying ${this.course}`);  
  }  
}
```

4. OOP Pillars in JS

Pillar	Meaning	JS Example
Encapsulation	Bundling data + behavior, hiding internal details	<code>#privateField</code> (true private fields), closures
Inheritance	Child class reuses parent's behavior	<code>extends</code> , <code>super</code>
Polymorphism	Same method name, different behavior in subclasses	Method overriding (<code>greet()</code> above)
Abstraction	Hiding complex implementation behind a simple interface	Exposing only public methods

```
class BankAccount {
  #balance = 0; // truly private (ES2022)
  deposit(amount) { this.#balance += amount; }
  getBalance() { return this.#balance; }
}
const acc = new BankAccount();
acc.deposit(100);
console.log(acc.#balance); // SyntaxError - cannot access private field
```

Interview Definition

“JavaScript classes are syntactic sugar over prototype-based inheritance, providing a cleaner syntax for constructors, methods, and inheritance.”

Interview Questions

1. Is JavaScript's `class` the same as classical OOP languages like Java?
2. What does `super()` do and why must it be called before `this`?
3. How do you create truly private fields in a JS class?
4. What is method overriding? Give an example.
5. How does `class` relate to `prototype`?

Interview Trap

Class methods are NOT own properties of instances — they live on the prototype, so `Object.keys(instance)` won't show them.

Real Project Usage

React class components (older style) used `extends React.Component`. Backend models (e.g., custom DB wrapper classes) commonly use ES6 classes for structuring logic in Node/Express projects.

Quick Revision

- `class` = sugar over prototypes, not a new mechanism.
- `super()` must run before `this` in a subclass constructor.

- `#field = true` private field (ES2022+).
 - 4 pillars: Encapsulation, Inheritance, Polymorphism, Abstraction.
-

24. JSON, FETCH API & HTTP BASICS

1. What is it?

- **JSON** (JavaScript Object Notation): a text-based data format used to exchange data between client and server.
- **Fetch API**: browser/Node built-in function to make HTTP requests.

2. JSON Methods

```
const obj = { name: "Mahil", age: 21 };  
const jsonStr = JSON.stringify(obj); // '{"name":"Mahil","age":21}'  
const backToObj = JSON.parse(jsonStr); // { name: "Mahil", age: 21 }
```

3. HTTP Methods

Method	Purpose
GET	Retrieve data
POST	Create new data
PUT	Replace an entire resource
PATCH	Update part of a resource
DELETE	Remove a resource

4. Common HTTP Status Codes

Code	Meaning
200	OK
201	Created
204	No Content
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
500	Internal Server Error

5. Fetch Examples

```
// GET
async function getUsers() {
  const res = await fetch("https://api.example.com/users");
  const data = await res.json();
  return data;
}

// POST
async function createUser(user) {
  const res = await fetch("https://api.example.com/users", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(user)
  });
  if (!res.ok) throw new Error(`HTTP error: ${res.status}`);
  return res.json();
}
```

Interview Questions

1. Difference between `JSON.stringify()` and `JSON.parse()`?
2. Difference between PUT and PATCH?
3. Does `fetch()` reject on a 404 response? (No! Only on network failure — you must check `res.ok` manually.)
4. What header is needed when sending JSON in a POST request?
5. What are the main HTTP status code categories (2xx, 4xx, 5xx)?

Interview Trap

`fetch()` does **NOT** throw an error for HTTP error statuses like 404 or 500 — it only rejects on network failure (e.g., no internet). You must manually check `response.ok` or `response.status`.

Real Project Usage

This is literally the client-side half of every MERN API call: React component calls `fetch()/axios` → Express API responds with JSON → React updates state with the parsed data.

Quick Revision

- `JSON.stringify` (JS → string) / `JSON.parse` (string → JS).
 - GET/POST/PUT/PATCH/DELETE map to CRUD operations.
 - `fetch()` only rejects on network errors — always check `res.ok`.
 - 2xx = success, 4xx = client error, 5xx = server error.
-

25. JAVASCRIPT CONCEPTS YOU MUST KNOW BEFORE REACT

React is “just JavaScript” — almost every core JS concept from Parts 1–3 shows up directly in React code.

JS Concept	Where it shows up in React
<code>map()</code>	Rendering lists: <code>items.map(item => <li key={item.id}>{item.name})</code>
<code>filter()</code>	Removing items from a list (e.g., delete a todo before re-rendering)
Destructuring Spread operator	Props: <code>function Card({ title, price }) {}</code> Immutable state updates: <code>setUser({ ...user, age: 22 })</code>
Objects & Arrays	Almost all component state/props are objects or arrays
Callbacks	Event handlers: <code><button onClick={() => handleClick(id)}></code>
Arrow functions	Used everywhere for inline handlers and short functions
Promises / <code>async-await</code>	Data fetching inside <code>useEffect</code>
Modules	<code>import React from "react"</code> , component files
Closures	Custom hooks “remember” state across renders
Immutability	Never mutate state directly — always create a new array/object

Example — Rendering a List

```
function TodoList({ todos, onDelete }) {  
  return (  
    <ul>  
      {todos.map(todo => (  
        <li key={todo.id}>  
          {todo.text}  
          <button onClick={() => onDelete(todo.id)}>Delete</button>  
        </li>  
      ))}  
    </ul>  
  )  
}
```

```
);  
}
```

Example — Immutable State Update

```
function App() {  
  const [todos, setTodos] = useState([]);  
  
  function addTodo(text) {  
    setTodos([...todos, { id: Date.now(), text }]); // spread - never push() directly!  
  }  
  
  function deleteTodo(id) {  
    setTodos(todos.filter(t => t.id !== id)); // filter - never splice() directly!  
  }  
}
```

Example — Data Fetching

```
useEffect(() => {  
  async function loadData() {  
    const res = await fetch("/api/todos");  
    const data = await res.json();  
    setTodos(data);  
  }  
  loadData();  
}, []); // empty dependency array = run once on mount
```

Interview Questions

1. Why is immutability important when updating React state?
2. Why do you need a **key** prop when rendering a list with `.map()`?
3. Why does data fetching usually go inside `useEffect`?
4. How do closures relate to custom React hooks?
5. Why use `filter()` instead of `splice()` to remove an item from state?

Interview Trap

Directly mutating state (`todos.push(newTodo); setTodos(todos)`) often does NOT trigger a re-render, because React compares object references — if the reference is the same, React assumes nothing changed. Always create a NEW array/object.

Quick Revision

- React state updates must be immutable (spread/filter/map, never push/splice/mutate directly).
- `.map()` → render lists, needs a stable **key**.
- Data fetching = `async/await` + `useEffect`.

- Everything you learned about closures directly explains how hooks “remember” values.
-

26. JAVASCRIPT FOR NODE.JS/EXPRESS

Node.js is JavaScript running outside the browser (via the V8 engine), and Express is a minimal web framework built on top of it.

Key Concepts in Backend JS

```
// Modules (CommonJS)
const express = require("express");
const app = express();

app.use(express.json()); // middleware - parses JSON body into req.body

// Route handler = a callback function
app.get("/users/:id", async (req, res) => {
  try {
    const { id } = req.params; // destructuring
    const user = await User.findById(id); // async/await + Promise
    if (!user) return res.status(404).json({ message: "Not found" });
    res.json(user); // JSON response
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.listen(3000, () => console.log("Server running"));
```

Middleware Concept

A middleware function runs BETWEEN the request and the final response, and can inspect/modify req/res or stop the chain.

```
function logger(req, res, next) {
  console.log(`${req.method} ${req.url}`);
  next(); // MUST call next() to pass control onward, or the request hangs forever
}
app.use(logger);
```

Error-Handling Middleware

```
app.use((err, req, res, next) => { // 4 params = Express recognizes this as error handler
  res.status(500).json({ error: err.message });
});
```

Interview Questions

1. What is middleware in Express, and why must you call `next()`?
2. Why is `async/await` used so heavily in Express route handlers?
3. What does `req.params` vs `req.body` vs `req.query` each contain?
4. Why does an Express error-handling middleware need exactly 4 parameters?
5. What happens if you forget to call `next()` in a middleware?

Interview Trap

Forgetting `next()` in a middleware causes the request to **hang forever** (no timeout, no error) — a very common beginner bug.

Real Project Usage

This is literally the backend half of MERN: React sends a `fetch` request → Express route handler (callback) → middleware chain runs → async DB query via Mongoose → JSON response sent back.

Quick Revision

- Route handlers = callbacks; almost always `async` for DB calls.
 - Middleware = functions that can inspect/modify the request; must call `next()`.
 - `req.params` (URL segments), `req.query` (query string), `req.body` (payload, needs `express.json()`).
 - Centralized error middleware needs `(err, req, res, next)` — 4 args, always.
-

27. JAVASCRIPT CONCEPTS USED IN MERN PROJECTS — FULL DATA FLOW

React (Frontend)

user clicks a button → event handler fires

fetch()/axios (async/await, Promise)

sends HTTP request (GET/POST/PUT/DELETE) with JSON body

Express (Node.js backend)

middleware chain (auth, JSON parsing, logging)

route handler (callback, async/await)

Mongoose (talks to MongoDB)

async queries return Promises

MongoDB (Database)

returns documents

Express sends `res.json(data)` back

React receives response (`await res.json()`)

`setState(data)` - immutable state update

React re-renders UI with new data (`map()` over the array)

JS Concepts Involved at Every Stage

Stage	JS Concepts
React → Event handler	Callbacks, arrow functions, closures
React → API call	<code>fetch</code> , Promises, <code>async/await</code> , JSON
Express → Middleware	Higher-order functions, callbacks
Express → Route handler	Destructuring (<code>req.body</code>), <code>async/await</code> , error handling
Mongoose → DB query	Promises (every Mongoose method returns one)

Stage	JS Concepts
Express → Response	JSON.stringify internally via res.json()
React → State update	Spread/rest, immutability, array methods (map, filter)
React → Re-render	map() with key prop

Interview Questions (Project-Based)

1. **“Why did you use async/await in your project?”** → “To handle asynchronous operations like API calls and database queries in a readable, sequential style, with `try/catch` for clean error handling.”
2. **“Why did you use map() here?”** → “To transform an array of data from the API into an array of React components, since `map()` returns a new array without mutating the original data.”
3. **“Why did you use spread syntax?”** → “To update state immutably — creating a new object/array instead of mutating the existing one, which React needs to detect changes and re-render correctly.”
4. **“How did you handle API errors?”** → “Using `try/catch` around `await fetch()` calls, checking `response.ok`, and showing appropriate error messages/states in the UI.”
5. **“How does your frontend communicate with your backend?”** → “Via HTTP requests using `fetch`/axios, sending and receiving JSON, with the Express backend exposing REST API routes.”
6. **“Why do you need JSON?”** → “It’s a lightweight, language-independent format both JavaScript objects and HTTP can work with directly, making it the standard for API communication.”
7. **“How did you avoid mutating state?”** → “By always using spread operators, `map()`, and `filter()` instead of `push()`/`splice()`/direct assignment on state variables.”

Quick Revision

- Memorize this data-flow diagram — it’s asked in nearly every MERN interview as “walk me through your project architecture.”
- Every stage maps back to a JS fundamental you already know from Parts 1–3.

28. JAVASCRIPT + DSA/CODING PATTERNS

JS Syntax Needed for DSA

```
// Arrays as stacks/queues
const stack = [];
stack.push(1); stack.pop();           // stack: LIFO
const queue = [];
queue.push(1); queue.shift();         // queue: FIFO (shift is O(n), better to use a real queue imp)

// Map for frequency counting
const freq = new Map();
freq.set("a", (freq.get("a") || 0) + 1);

// Set for uniqueness
const seen = new Set();

// Sorting
arr.sort((a, b) => a - b); // ascending numeric sort - default sort() is STRING-based!

// Recursion
function factorial(n) {
  if (n <= 1) return 1;
  return n * factorial(n - 1);
}
```

Interview Trap: `arr.sort()` with NO comparator sorts elements as **strings** — `[10, 1, 2].sort()` gives `[1, 10, 2]`, not `[1, 2, 10]`. Always pass `(a,b) => a-b` for numeric sorts.

1. Frequency Map Pattern

Idea: Count occurrences of elements using an object/Map.

```
function charFrequency(str) {
  const freq = {};
  for (const ch of str) freq[ch] = (freq[ch] || 0) + 1;
  return freq;
}
```

Easy problem: First non-repeating character in a string. **Medium problem:** Group anagrams together using sorted-string as a frequency key.

2. Two Pointer Pattern

Idea: Use two indices moving toward each other (or in the same direction) to avoid nested loops.

```
function isPalindrome(s) {
  let left = 0, right = s.length - 1;
  while (left < right) {
    if (s[left] !== s[right]) return false;
    left++; right--;
  }
  return true;
}
```

Easy problem: Check if a string is a palindrome. **Medium problem:** Find two numbers in a sorted array that sum to a target.

3. Sliding Window Pattern

Idea: Maintain a “window” (subarray/substring) and expand/shrink it instead of recomputing from scratch.

```
function maxSubarraySum(arr, k) {
  let maxSum = 0, windowSum = 0;
  for (let i = 0; i < arr.length; i++) {
    windowSum += arr[i];
    if (i >= k - 1) {
      maxSum = Math.max(maxSum, windowSum);
      windowSum -= arr[i - (k - 1)];
    }
  }
  return maxSum;
}
```

Easy problem: Max sum subarray of size k. **Medium problem:** Longest substring without repeating characters.

4. Hashing Pattern

Idea: Use a Map/object for $O(1)$ lookups instead of nested loops ($O(n^2) \rightarrow O(n)$).

```
function twoSum(nums, target) {
  const map = new Map();
  for (let i = 0; i < nums.length; i++) {
    const complement = target - nums[i];
    if (map.has(complement)) return [map.get(complement), i];
    map.set(nums[i], i);
  }
}
```

```
}  
}
```

Easy problem: Two Sum. **Medium problem:** Subarray sum equals K.

5. Stack Pattern

Idea: LIFO structure — great for matching pairs, undo operations, expression evaluation.

```
function isValidParens(s) {  
  const stack = [];  
  const map = { ")": "(", "]" : "[", "}": "{" };  
  for (const ch of s) {  
    if (!map[ch]) stack.push(ch);  
    else if (stack.pop() !== map[ch]) return false;  
  }  
  return stack.length === 0;  
}
```

Easy problem: Valid parentheses. **Medium problem:** Next greater element.

6. Queue Pattern

Idea: FIFO structure — used in BFS (Breadth-First Search) and order-preserving processing.

```
function bfs(graph, start) {  
  const visited = new Set([start]);  
  const queue = [start];  
  const order = [];  
  while (queue.length) {  
    const node = queue.shift();  
    order.push(node);  
    for (const neighbor of graph[node] || []) {  
      if (!visited.has(neighbor)) {  
        visited.add(neighbor);  
        queue.push(neighbor);  
      }  
    }  
  }  
  return order;  
}
```

Easy problem: Level order traversal of a tree. **Medium problem:** Shortest path in an unweighted graph (BFS).

7. Binary Search Pattern

Idea: Repeatedly halve a SORTED search space — $O(\log n)$.

```
function binarySearch(arr, target) {  
  let low = 0, high = arr.length - 1;
```

```

while (low <= high) {
  const mid = Math.floor((low + high) / 2);
  if (arr[mid] === target) return mid;
  if (arr[mid] < target) low = mid + 1;
  else high = mid - 1;
}
return -1;
}

```

Easy problem: Search in a sorted array. **Medium problem:** Find first/last position of an element (search for a range).

8. Prefix Sum Pattern

Idea: Precompute cumulative sums to answer range-sum queries in $O(1)$ instead of $O(n)$ each time.

```

function buildPrefixSum(arr) {
  const prefix = [0];
  for (let i = 0; i < arr.length; i++) prefix.push(prefix[i] + arr[i]);
  return prefix; // rangeSum(l, r) = prefix[r+1] - prefix[l]
}

```

Easy problem: Range sum query (immutable array). **Medium problem:** Subarray sum equals K (combined with hashing).

9. Recursion Pattern

Idea: Break a problem into smaller identical sub-problems, with a clear base case.

```

function fibonacci(n) {
  if (n <= 1) return n; // base case
  return fibonacci(n - 1) + fibonacci(n - 2); // recursive case
}

```

Easy problem: Factorial / Fibonacci. **Medium problem:** Generate all subsets (power set) or permutations of an array.

10. Sorting Pattern

Idea: Many problems become trivial once the array is sorted first.

```
arr.sort((a, b) => a - b);
```

Easy problem: Sort an array of 0s, 1s, and 2s (Dutch National Flag — or just `.sort()`). **Medium problem:** Merge overlapping intervals (sort by start time first).

Interview Questions

1. Why does `arr.sort()` behave oddly on numbers without a comparator?
2. When would you choose a Map/hashing approach over nested loops?
3. What's the time complexity improvement from prefix sums for range-sum queries?

4. Why is BFS implemented with a queue and DFS with a stack (or recursion)?
5. What's the base case in recursion, and why is it required?

Quick Revision

- Always pass a comparator to `.sort()` for numbers.
 - Two Pointer / Sliding Window $\rightarrow$ array/string problems, avoid nested loops.
 - Hashing (Map/Set) $\rightarrow$ turns $O(n^2)$ lookups into $O(n)$.
 - Stack $\rightarrow$ matching/undo problems. Queue $\rightarrow$ BFS/level-order problems.
 - Binary Search needs a SORTED array — $O(\log n)$.
 - Prefix Sum $\rightarrow$ precompute once, answer range queries in $O(1)$.
-

29. JAVASCRIPT INTERVIEW QUESTION BANK

Beginner (18)

1. **What are the data types in JavaScript?** → 7 primitives (string, number, boolean, undefined, null, symbol, bigint) + object (includes arrays, functions).
2. **Difference between var, let, const?** → Scope (function vs block) and reassignment/redeclaration rules (Ch. 2).
3. **What is hoisting?** → Declarations moved to the top of scope during compile phase (Ch. 2).
4. **What is the difference between == and ===?** → Loose (coerces types) vs strict (checks type + value) (Ch. 3).
5. **What is NaN?** → Result of invalid math; `typeof NaN === "number"`; check with `Number.isNaN()`.
6. **What is a function expression vs declaration?** → Expression not hoisted with its body; declaration is (Ch. 4).
7. **What are arrow functions and how are they different?** → Shorter syntax, no own `this`/arguments (Ch. 4, 5).
8. **What is the DOM?** → Tree representation of an HTML page (Ch. 11).
9. **What is null vs undefined?** → Intentional absence vs unassigned (Ch. 1).
10. **What is a template literal?** → Backtick strings with `${}` interpolation.
11. **What is the difference between slice() and splice()?** → Non-mutating extraction vs mutating insert/remove (Ch. 7).
12. **What is destructuring?** → Unpacking array/object values into variables (Ch. 9).
13. **What is the spread operator?** → Expands an iterable into individual elements (Ch. 9).
14. **What is an array method that transforms every element?** → `map()`.
15. **What is typeof used for?** → Checking a value's type at runtime.
16. **What is a callback function?** → A function passed to run later (Ch. 13).
17. **What does JSON.stringify() do?** → Converts a JS object to a JSON string (Ch. 24).
18. **What is the difference between a parameter and an argument?** → Parameter = variable in function definition; argument = actual value passed at call time.

Intermediate (22)

19. **Explain closures with an example.** → Ch. 6.
20. **Explain the Event Loop.** → Ch. 17 (this is the single most important answer to have ready).
21. **Difference between microtasks and macrotasks?** → Ch. 18.
22. **How does this behave differently in arrow vs regular functions?** → Ch. 5.

23. **What is the difference between `Promise.all()` and `Promise.allSettled()`?** → Ch. 15.
24. **What is event delegation and why use it?** → Ch. 12.
25. **What is the prototype chain?** → Ch. 22.
26. **Explain `call()`, `apply()`, and `bind()`.** → Ch. 5.
27. **What is a higher-order function?** → Takes and/or returns a function (Ch. 4).
28. **What's the difference between shallow copy and deep copy?** → `Spread/Object.assign` = shallow; `structuredClone()`/deep-clone libs = deep (Ch. 8/9).
29. **What is the Temporal Dead Zone?** → Ch. 2.
30. **How do you handle errors in `async/await`?** → `try/catch` (Ch. 16, 19).
31. **What is a Set and how is it different from an array?** → Only unique values, no indexing (Ch. 20).
32. **What is a Map and when would you use it over an object?** → Any key type, guaranteed order, `.size` (Ch. 20).
33. **What is optional chaining and why is it useful?** → Ch. 20.
34. **What is the difference between `for...of` and `for...in`?** → Ch. 20.
35. **What are ES modules vs CommonJS?** → Ch. 21.
36. **Explain middleware in Express.** → Ch. 26.
37. **How would you debounce a function, and why?** → Delay execution until a pause in calls (uses closures + `setTimeout`); used to limit API calls on every keystroke in a search box.
38. **What is immutability and why does React care about it?** → Ch. 25.
39. **What is the difference between `Object.freeze()` and `const`?** → `const` prevents reassignment of the variable; `Object.freeze()` prevents mutation of the object's properties (shallow).
40. **What does the `key prop` do in React lists?** → Helps React efficiently identify which items changed/added/removed.

Advanced (15)

41. **Walk through exactly what happens in the event loop for a mix of `setTimeout` and Promises.** → Ch. 17.
42. **How would you implement your own simplified `Promise.all()`?** → Track resolved count, resolve when all settle, reject on first rejection.
43. **Explain currying and give an example.** → Transforming a function of multiple arguments into a sequence of functions each taking one argument: `const add = a => b => a + b;`
44. **What's the difference between `debounce` and `throttle`?** → `Debounce` delays until calls stop; `throttle` limits to once per interval regardless of call frequency.
45. **How does garbage collection work in JS (conceptually)?** → Mark-and-sweep: unreachable objects (no references) get collected automatically.
46. **What is memoization and how would you implement it?** → Caching function results by input, often via a closure-held Map.
47. **Explain the difference between deep equality and reference equality for objects.** → `===` compares references; deep equality requires manual/library comparison of all nested values.
48. **What is a pure function?** → Same input always gives same output, no side effects — foundational to React's rendering model.
49. **How would you cancel a `fetch` request?** → Using `AbortController`.
50. **What's the difference between `Object.freeze()` (shallow) vs a fully immutable nested object?** → Freeze only locks the top level; nested objects remain mutable unless recursively frozen.
51. **How does `async/await` interact with the event loop internally?** → `await` pauses the

function and its continuation is scheduled as a microtask once the awaited promise settles.

- 52. **What is tree-shaking and how do ES modules enable it?** → Removing unused exported code at build time; possible because ES module imports/exports are statically analyzable (Ch. 21).
 - 53. **What's a WeakMap and why would you use one?** → Like Map but keys must be objects and are weakly referenced, allowing garbage collection — useful for caching without memory leaks.
 - 54. **How would you flatten a deeply nested array without using `.flat()`?** → Recursion, checking `Array.isArray()` on each element.
 - 55. **What's the difference between synchronous and asynchronous recursion in terms of stack usage?** → Synchronous recursion can overflow the call stack for deep recursion; async recursion (via `setTimeout/promises`) yields control and avoids stack overflow but is slower.
-

30. JAVASCRIPT OUTPUT-BASED QUESTIONS

Level 1 — Basic

Q1.

```
console.log(1 + "1");  
console.log("5" - 2);
```

Output: "11", 3 — + with a string concatenates; - coerces both sides to numbers.

Q2.

```
console.log(typeof null);  
console.log(typeof undefined);  
console.log(typeof NaN);
```

Output: "object", "undefined", "number" — memorize these three (Ch. 1).

Q3.

```
console.log([] == false);
```

Output: true — [] is coerced to "", then to 0, and false is coerced to 0. Classic == weirdness.

Level 2 — Scope/Hoisting

Q4.

```
console.log(a);  
var a = 5;
```

Output: undefined — var is hoisted and initialized as undefined (Ch. 2).

Q5.

```
console.log(b);  
let b = 5;
```

Output: ReferenceError — TDZ (Ch. 2).

Q6.

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 0);  
}
```

Output: 3 3 3 — var shares one variable across all iterations (Ch. 6).

Q7.

```
for (let i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 0);  
}
```

Output: 0 1 2 — let creates a new binding per iteration (Ch. 6).

Level 3 — Functions/Closures

Q8.

```
function outer() {  
  let count = 0;  
  return () => ++count;  
}  
const counter = outer();  
console.log(counter());  
console.log(counter());
```

Output: 1, 2 — closure retains count across calls (Ch. 6).

Q9.

```
console.log(sum(5));  
function sum(a) { return a + 1; }
```

Output: 6 — function declarations are fully hoisted (Ch. 2, 4).

Q10.

```
console.log(mul(5));  
var mul = function(a) { return a * 2; };
```

Output: TypeError: mul is not a function — only the var declaration is hoisted (as undefined), not the function body (Ch. 2, 4).

Level 4 — this

Q11.

```
const obj = {  
  name: "X",  
  greet: function() { console.log(this.name); }  
};  
const fn = obj.greet;  
fn();
```

Output: undefined (or error in strict mode) — fn is called standalone, losing obj as its this (Ch. 5).

Q12.

```
const obj = {
  name: "X",
  greet: () => console.log(this.name)
};
obj.greet();
```

Output: undefined — arrow function's this comes from the surrounding (module/global) scope, not obj (Ch. 5).

Q13.

```
function Person(name) { this.name = name; }
Person.prototype.greet = function() { console.log(this.name); };
const p = new Person("Mahil");
const greetFn = p.greet;
greetFn();
```

Output: undefined — same detachment issue as Q11.

Level 5 — Async/Promises

Q14.

```
console.log("1");
setTimeout(() => console.log("2"), 0);
console.log("3");
```

Output: 1 3 2 — sync code always runs before macrotasks (Ch. 17).

Q15.

```
Promise.resolve(1).then(v => console.log(v));
console.log(2);
```

Output: 2 1 — .then() is a microtask, runs after all sync code (Ch. 17).

Q16.

```
async function f() {
  console.log("a");
  await null;
  console.log("b");
}
f();
console.log("c");
```

Output: a c b — code before await runs sync; everything after await becomes a microtask (Ch. 16, 17).

Q17.

```

Promise.resolve().then(() => console.log("1"));
Promise.resolve().then(() => console.log("2"));
setTimeout(() => console.log("3"), 0);
Promise.resolve().then(() => console.log("4"));

```

Output: 1 2 4 3 — all microtasks (Promises) drain fully before the macrotask (`setTimeout`) (Ch. 17, 18).

Level 6 — Event Loop (multi-step)

Q18.

```

console.log("start");
setTimeout(() => console.log("timeout"), 0);
Promise.resolve()
  .then(() => console.log("promise 1"))
  .then(() => console.log("promise 2"));
console.log("end");

```

Output: start end promise 1 promise 2 timeout **Step-by-step:** sync start/end first → both `.then()`s are microtasks and chain sequentially before the macrotask `setTimeout` runs.

Q19.

```

async function async1() {
  console.log("A");
  await async2();
  console.log("B");
}
async function async2() { console.log("C"); }
console.log("D");
async1();
console.log("E");

```

Output: D A C E B **Step-by-step:** D (sync) → `async1()` called → A (sync, inside async fn) → `async2()` called → C (sync, runs immediately since it's not awaited yet) → control returns to `async1`, hits `await`, pauses, yields to caller → E (sync, back in main) → call stack empty → microtask resumes → B.

Q20.

```

setTimeout(() => console.log("1"), 0);
new Promise((resolve) => {
  console.log("2");
  resolve();
}).then(() => console.log("3"));
console.log("4");

```

Output: 2 4 3 1 — the Promise **executor function** itself runs synchronously (2), only `.then()` is a microtask; 4 (sync) runs before the microtask 3; `setTimeout`'s 1 is last (macrotask).

Bonus — Mixed Traps

Q21.

```
console.log(0.1 + 0.2 === 0.3);
```

Output: `false` — floating-point precision issue; `0.1 + 0.2` is actually `0.30000000000000004`.

Q22.

```
const arr = [1, 2, 3];
const arr2 = arr;
arr2.push(4);
console.log(arr);
```

Output: `[1, 2, 3, 4]` — arrays are reference types; `arr2` and `arr` point to the same array (Ch. 1).

Q23.

```
console.log([1, 2, 10].sort());
```

Output: `[1, 10, 2]` — default `.sort()` converts to strings (Ch. 28).

Q24.

```
let x = 10;
function test() {
  console.log(x);
  let x = 20;
}
test();
```

Output: `ReferenceError: Cannot access 'x' before initialization` — the inner `let x` shadows the outer `x` for the **WHOLE** function scope, including before its declaration line (TDZ applies even though an outer `x` exists) (Ch. 2).

31. COMMON JAVASCRIPT INTERVIEW TRAPS — “TRAPS INTERVIEWERS LOVE”

Trap	The Catch
<code>==</code> vs <code>===</code>	<code>==</code> coerces types before comparing; always prefer <code>===</code>
<code>null</code> vs <code>undefined</code>	Intentional absence vs uninitialized; both are falsy but <code>typeof</code> differs
<code>NaN</code>	<code>NaN !== NaN</code> ; use <code>Number.isNaN()</code>
<code>var</code> hoisting	Hoisted + initialized <code>undefined</code> , causing silent bugs instead of errors
<code>let/const</code> TDZ	Hoisted but inaccessible until declaration line — throws instead of <code>undefined</code>
Closures inside loops	<code>var</code> shares one binding; <code>let</code> creates a new one per iteration
<code>this</code> in arrow functions	No own <code>this</code> — inherits from enclosing lexical scope, breaking naive object-method use
<code>map()</code> vs <code>forEach()</code>	<code>map</code> returns a new array; <code>forEach</code> returns <code>undefined</code>
<code>slice()</code> vs <code>splice()</code>	<code>slice</code> = non-mutating extraction; <code>splice</code> = mutating insert/remove
Shallow vs deep copy	<code>Spread/Object.assign</code> only copy one level deep — nested objects still shared
Reference vs value	Primitives copy by value; objects/arrays copy by reference
Mutable vs immutable	Primitives immutable; objects/arrays mutable — React state relies on this distinction
Promise vs <code>async/await</code>	<code>async/await</code> is sugar over Promises — still non-blocking, not truly synchronous
Microtask vs macrotask	ALL microtasks drain before the next macrotask, always
<code>prototype</code> vs <code>__proto__</code>	<code>prototype</code> lives on constructor functions; <code>__proto__</code> lives on instances
Spread operator shallow copy	Same shallow-copy trap as <code>Object.assign()</code>
<code>fetch()</code> not rejecting on 404/500	Only rejects on network failure — must check <code>response.ok</code> manually

Trap	The Catch
<code>arr.sort()</code> without comparator <code> </code> vs <code>??</code>	Sorts numbers as strings by default <code> </code> treats ALL falsy values (0, "") as “missing”; <code>??</code> only checks <code>null/undefined</code>

32. FINAL JAVASCRIPT CHEAT SHEET (20–30 min revision)

Core Syntax

```
let, const           // block scope; var = function scope (avoid)
() => {}              // arrow function, no own `this`
`${var}`             // template literal
const {a,b} = obj     // destructuring
const [x,y] = arr
const merged = {...a, ...b} // spread (shallow copy)
function f(...args){} // rest - collects into array
```

Equality & Types

```
== → coerces (avoid)      === → strict (prefer)
typeof null → "object"    typeof NaN → "number"
Number.isNaN(x) → correct NaN check
```

Array Methods

```
map, filter          → new array, non-mutating
reduce                → single value
forEach               → no return value
find/findIndex        → first match / its index
some/every            → boolean checks
includes              → value exists?
sort/reverse/splice → MUTATE original
slice/concat/flat    → do NOT mutate
```

Object Methods

```
Object.keys/values/entries(obj)
Object.assign({}, obj) // shallow copy
{...obj}               // shallow copy
structuredClone(obj)   // deep copy
```

Functions & this

```
this = who called the function (except arrows = lexical)
```

call(obj, a, b) → invoke now, args individually
apply(obj, [a,b]) → invoke now, args as array
bind(obj) → returns new function, call later

Closures

```
function outer(){ let x=0; return ()=>++x; } // inner fn "remembers" x
```

Async

Promise states: pending → fulfilled/rejected (permanent)

.then/.catch/.finally

Promise.all → fail-fast, all results

Promise.allSettled → always waits, per-item status

Promise.race → first settled wins

Promise.any → first fulfilled wins

async function → always returns a Promise

await → pauses only that function

try/catch → wraps awaited errors

Event Loop (memorize this order!)

1. Run all synchronous code (Call Stack)
2. Drain ALL microtasks (Promise .then, queueMicrotask, async continuations)
3. Run ONE macrotask (setTimeout, setInterval, I/O)
4. Repeat from step 2

DOM & Events

querySelector / getElementById

addEventListener("click", handler)

event bubbling (default) vs capturing

event delegation → one listener on parent, check e.target

e.preventDefault() // stop default browser behavior

e.stopPropagation() // stop bubbling

Prototypes & Classes

obj.__proto__ === Constructor.prototype

```
class X { constructor(){ } method(){ } }
```

```
class Y extends X { constructor(){ super(); } }
```

class = syntactic sugar over prototypes

Modules

ES Modules: export / import (static, tree-shakeable)

CommonJS: module.exports / require (Node traditional)

Map / Set

`new Map()` → any key type, `.set/.get/.size`
`new Set()` → unique values, `[...new Set(arr)]` to dedupe

DSA Quick Syntax

Stack: push/pop Queue: push/shift
Map for frequency/hashing: $O(1)$ lookup
Two Pointer: left/right indices closing in
Sliding Window: expand/shrink a subarray
Binary Search: needs SORTED array, $O(\log n)$
Recursion: base case + recursive case

33. JAVASCRIPT — 1 DAY BEFORE INTERVIEW

MUST KNOW

- Event Loop (Ch. 17) — with the `console.log("A"); setTimeout(...); Promise.then(...); console.log("D");` example fully memorized and explainable.
- Closures (Ch. 6) — counter example + the `var`-in-loop trap and its `let` fix.
- `this` behavior — regular vs arrow, plus `call`/`apply`/`bind` (Ch. 5).
- Promises vs `async/await` (Ch. 15–16).
- `==` vs `===`, `null` vs `undefined`, NaN handling (Ch. 1, 3).
- `map`/`filter`/`reduce` and mutating vs non-mutating array methods (Ch. 7).
- Prototype chain + why `class` is sugar over it (Ch. 22–23).

SHOULD KNOW

- Event delegation & bubbling/capturing (Ch. 12).
- Spread/rest difference, shallow vs deep copy (Ch. 9).
- Middleware concept in Express + why `next()` matters (Ch. 26).
- MERN request/response data flow end-to-end (Ch. 27).
- DSA patterns: Two Pointer, Sliding Window, Hashing, Binary Search (Ch. 28).

IF TIME PERMITS

- WeakMap, currying, memoization, debounce vs throttle, tree-shaking.
- Custom error classes, `AbortController`.
- Private class fields (`#field`).

50 QUESTIONS I MUST BE ABLE TO ANSWER

→ See the full 55-question bank in Chapter 29 (Beginner 18 + Intermediate 22 + Advanced 15) — that set covers this.

20 OUTPUT QUESTIONS I MUST BE ABLE TO SOLVE

→ See the full 24-question set in Chapter 30 — walk through each explanation out loud once before the interview.

10 CODING PATTERNS I MUST REMEMBER

1. Frequency Map
2. Two Pointer
3. Sliding Window
4. Hashing
5. Stack
6. Queue (BFS)
7. Binary Search
8. Prefix Sum
9. Recursion
10. Sorting-first approach

End of Notebook. You now have all 5 parts covering fundamentals → async/event loop → OOP/prototypes/HTTP → React/Node/MERN/DSA → interview & revision material. Good luck with placements — ask me any time to expand a specific chapter, add more practice questions, or drill you with a quiz on any topic.